— Informed Intrinsically Interpretable Machine Learning (IIIML)—

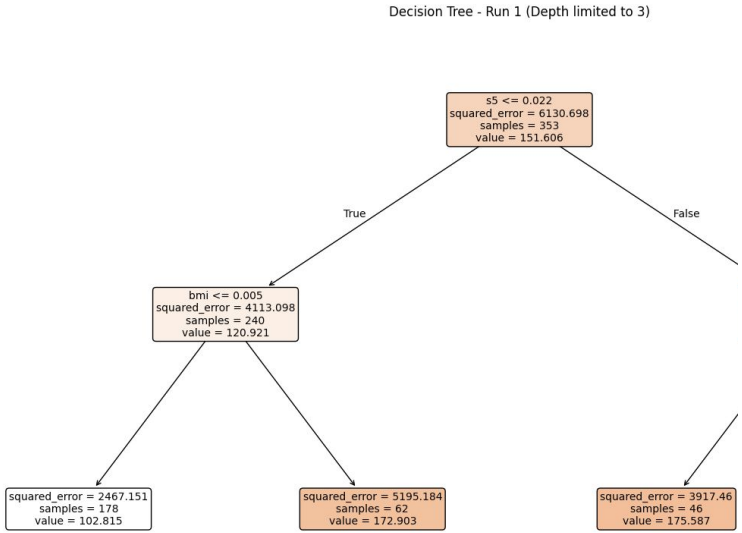
Anastasia Samets & Seyed-Alireza Miri & Mathias Kraus

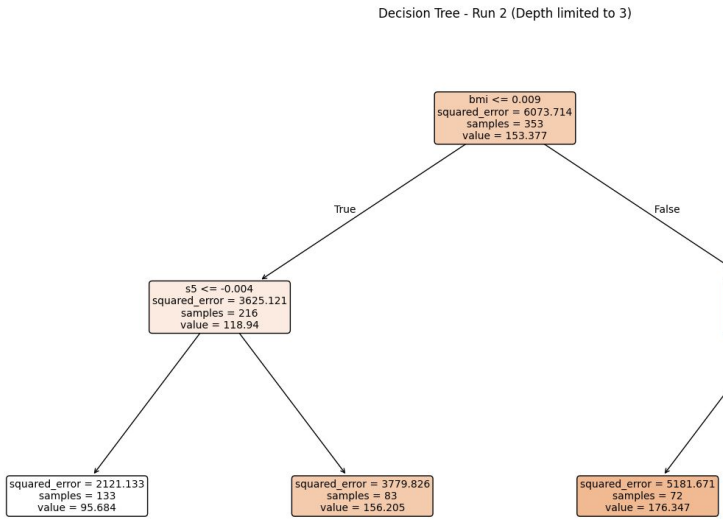
Universität Regensburg

Agenda

1. Intro Informed ML
2. Intro Gurobi
3. Examples in Gurobi
    a. Sign-Constrained Lin. Reg.
    b. Importance + Sign-Constrained Lin. Reg.
    c. Importance-Constrained Integer Lin. Reg.
    d. Monotonically Constrained Extreme Learning Machine
4. Informed IGANN
5. Optimal Decision Trees
6. Possible Research Contributions

Motivation

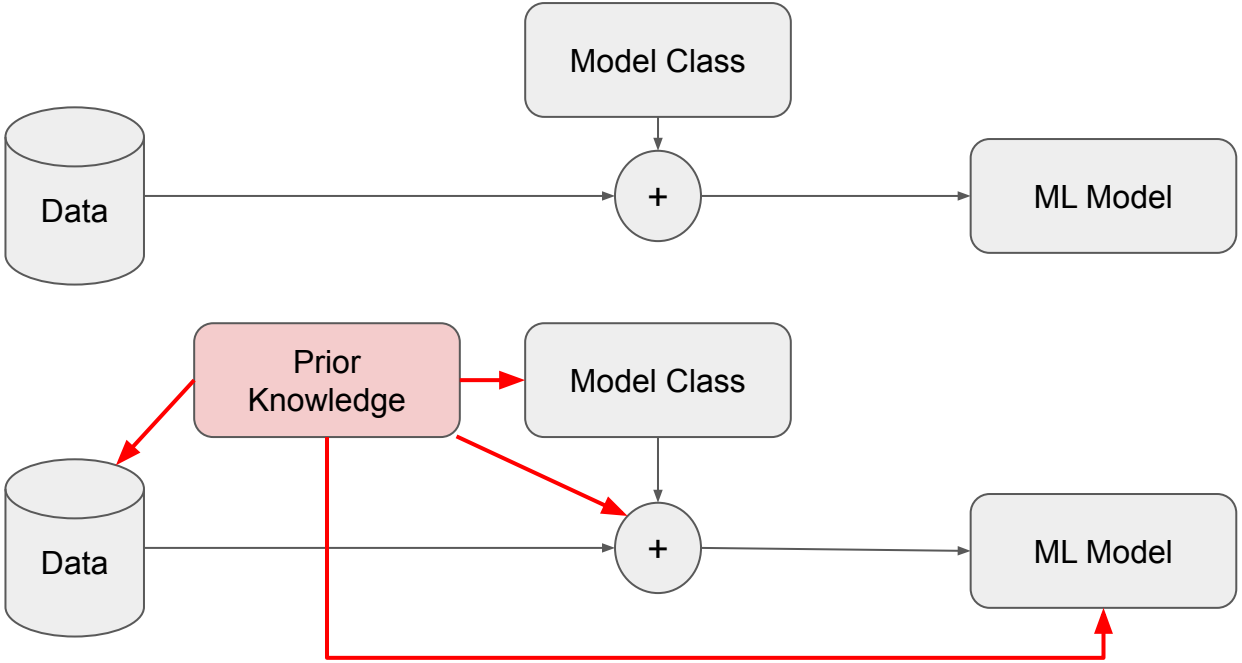




Model classes as simple as the decision tree lead to <u>completely</u> different models. This will likely confuse users and make them less likely to use the model.
→ What we want is some way to "tighten" the search space for the model.

What is Informed Machine Learning?

Definition: Informed machine learning describes learning from a hybrid information source that consists of data and prior knowledge.



Prior knowledge, e.g.,
• Information on physical laws • Information that constrain model behavior for a better fit • Potential simplifications that increase acceptance of users

Why do We Need Informed Machine Learning?

Technical	
Better Performance	Prior knowledge can help machine learning models achieve superior predictive accuracy or generalizability, particularly when dealing with complex problems or noisy data
Training with Less Data	Overcome scenarios where insufficient training data is available. Prior knowledge acts as an additional information source to guide the learning process.
Reducing Model Search Space	Integrating prior knowledge can significantly constrain the hypothesis set, thereby reducing the complexity of the learning problem and making it more efficient.
Human	
Ensuring Mental Model Alignment	Models align with user knowledge.
Ensuring Regulatory Alignment	Models align with natural laws, or regulatory guidelines.

Taxonomy/Characterization of Informed ML

Knowledge Source

- *Scientific Knowledge* (science, technology, engineering, and mathematics. Typically formalized and validated through scientific experiments)
- *World Knowledge* (refers to common facts from everyday life, or general knowledge)
- ***Expert Knowledge*** (knowledge held by experts. It is often more informal and implicitly validated within the expert community, requiring formalization via human-machine interfaces

Knowledge Representation

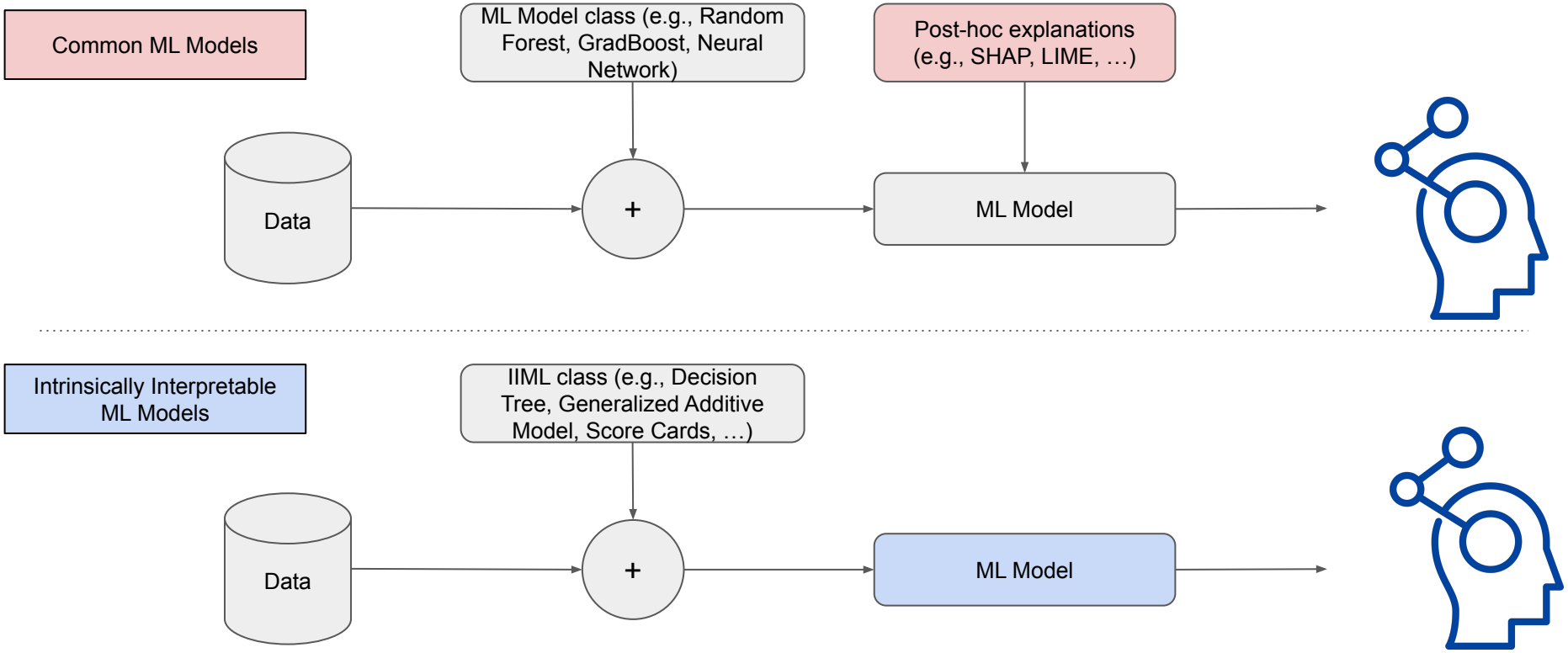
- ***Algebraic Equations*** (Represent knowledge as equality or inequality relations between mathematical expressions, often used to describe functions or constrain variables)
- ***Human Feedback*** (Refers to technologies that transform knowledge via direct interfaces between users and machines, often less formalized and stemming from expert intuition)
- *Differential Equations*
- *Simulation Results*
- …

Knowledge Integration

- *Training Data* (creating a second dataset from a separate source of prior knowledge)
- *Hypothesis Set* (Knowledge is integrated by defining the model structure, e.g., Knowledge-Based Artificial Neural Networks - KBANNs)
- ***Learning Algorithm*** (modifying the learning algorithm, e.g., the loss function)
- ***Final Hypothesis*** (Output of the learning pipeline (the final hypothesis) is benchmarked or validated against existing knowledge)

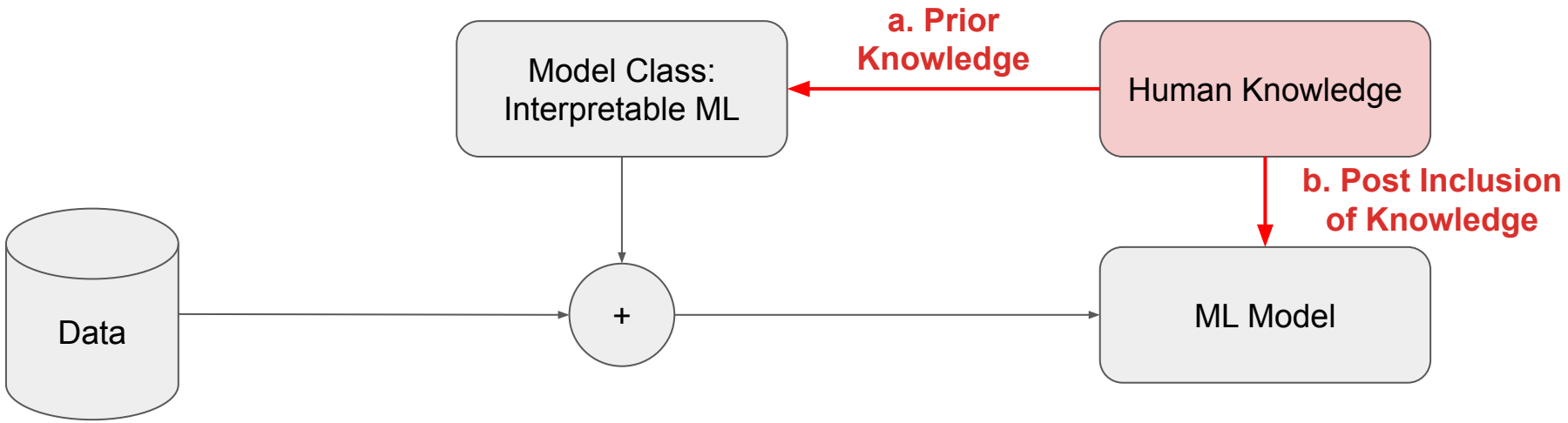
Von Rueden, L., Mayer, S., Beckh, K., Georgiev, B., Giesselbach, S., Heese, R., ... & Schuecker, J. (2021). Informed machine learning–a taxonomy and survey of integrating prior knowledge into learning systems. IEEE Transactions on Knowledge and Data Engineering, 35(1), 614-633.

Intrinsically Interpretable Machine Learning (IIML)



In IIML, we <u>directly</u> analyse the ML model, without any proxy in-between.

(Human) Informed Intrinsically Interpretable Machine Learning (IIIML)



a. Use prior knowledge to inform the model class (which hypothesis/models can be obtained)
b. Use knowledge to correct IIML models (does the hypothesis agree with my view/mental model)

Getting Knowledge into an ML Model

“Modern” Way

- Formulate the condition as a soft-constrained and add it as regularization when training your ML model
- E.g., monotonic constrained of a feature `x` in PyTorch

$$\frac{\partial \hat{y}}{\partial x_i} \geq 0$$

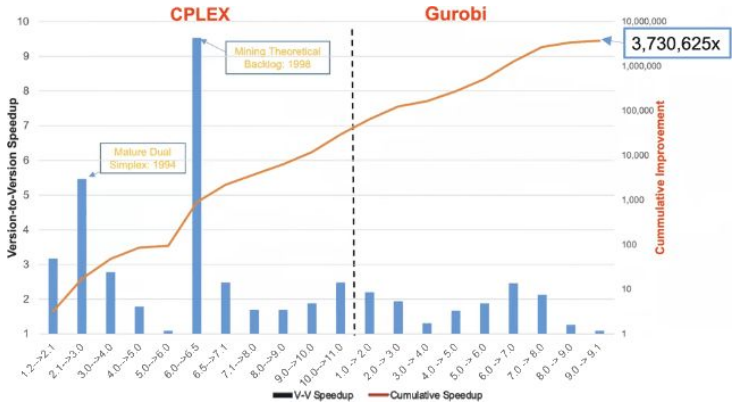
```
grad = torch.grad.grad(
    outputs=y_pred,
    inputs=x,
    ...
)
...
penalty = torch.clamp(-grad[:,feature_index],
min=0).mean()
total_loss = loss + reg_strength * penalty
```

“Old-fashioned” Way

- Formulate the condition as a hard constrained and solve the problem optimally using mathematical optimization packages
  - Linear/Quadratic/Convex/Mixed Integer/… Optimization
  - with e.g., Linear/Quadratic Constraints
- The constraints will be guaranteed

Development of Mathematical Optimization

- Solving problems optimally is hard
- That's why many developments in ML use non-optimal solvers, or heuristics
- However, for more and more problems, the development of mathematical optimization now allows solving them optimally due to algorithmic and hardware improvements
- Probably, this also includes many IIML models (decision trees, extreme learning machines, scoring models, …)



Combining the computer hardware speed increase of 4,000 times with the solver software performance improvement of 5 million times, the total improvement **from 1989 to 2024 is a factor of 20 billion times faster!**

At that rate, a model that takes 60 seconds to solve in 2023 would have taken 38,000 years to solve if started in 1989.

Combining software and hardware improvements, many problems can be solved 10.000 - 100.000 times faster than 10 years ago. <u>The problems may seem difficult, but they are solvable nowadays.</u> Dimitris Bertsimas (OR, MIT) calls this practical tractability.

Gurobi

- Gurobi and CPLEX are by far the most widely used solvers in both academia and industry.
- Gurobi offers a free academic license: simply register, sign up for a WLS license (or any other), download the license file and put it in your home.
- pip install gurobipy (this also works without a license, but then only “smaller” models will be solved)
- Almost all choices for best optimization etc. will be made by Gurobi
- So you basically design your problem and throw it into the solver…

Sign Constrained Linear Regression using Gurobi

```
import numpy as np
import gurobipy as gp
from gurobipy import GRB

np.random.seed(42)
n, p = 100, 3
X = np.random.randn(n, p)
y = X @ np.array([2.0, -3.0, 1.0]) + 0.5*np.random.randn(n)

m = gp.Model("lin reg")
beta = m.addMVar(shape=p, lb=-GRB.INFINITY, name="beta")

residual = y - X @ beta
m.setObjective(residual @ residual)

m.addConstr(beta[0] >= 0)
m.addConstr(beta[1] >= 0)
m.addConstr(beta[2] >= 0)

m.optimize()
print(beta.X)

>>> [2.27482401e+00 2.48676814e-11 1.33735573e+00]
```

Importance and Signed Constrained Linear Regression in Gurobi

```
import numpy as np
import gurobipy as gp
from gurobipy import GRB

np.random.seed(42)
n, p = 100, 3
X = np.random.randn(n, p)
y = X @ np.array([2.0, -3.0, -1.0]) + 0.5*np.random.randn(n)

m = gp.Model("lin reg")
beta = m.addMVar(shape=p, lb=-GRB.INFINITY, name="beta")

residual = y - X @ beta
m.setObjective(residual @ residual)

m.addConstr(beta[0] >= 0)
m.addConstr(beta[1] >= 0)
m.addConstr(beta[2] <= 0)
m.addConstr(beta[2] * beta[2] >= beta[0] * beta[0])

m.optimize()
print(beta.X)

>>> [ 1.25959266e+00  3.08493677e-14 -1.25959266e+00]
```

$beta^2 >= beta^0$

Importance Constrained Linear Integer Regression in Gurobi

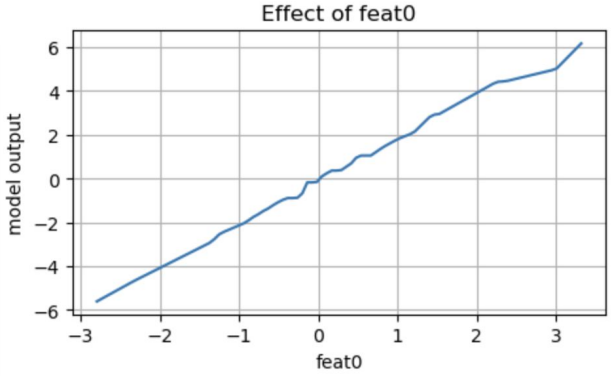
```
import numpy as np
import gurobipy as gp
from gurobipy import GRB

np.random.seed(42)
n, p = 100, 3
X = np.random.randn(n, p)
y = X @ np.array([-1.3, 3.2, 0.7]) + 0.5 * np.random.randn(n)

m = gp.Model("integer_lin_reg")

beta = m.addMVar(shape=p, vtype=GRB.INTEGER, name="beta")

residual = y - X @ beta
m.setObjective(residual @ residual, GRB.MINIMIZE)

m.addConstr(beta[2] * beta[2] >= beta[0] * beta[0],
            name="beta2_ge_beta0_abs")

m.optimize()

print("Optimal integer beta:", beta.X)
>>> Optimal integer beta: [0. 3. 1.]
```

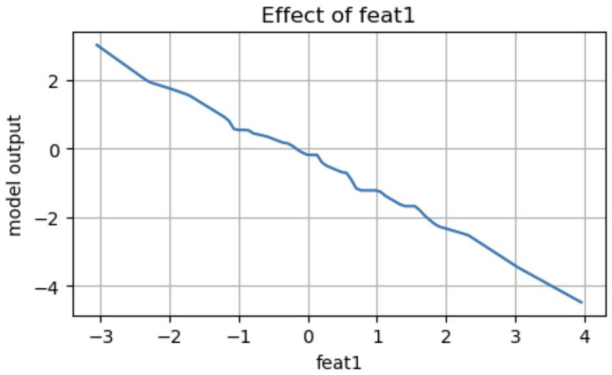
Coefficients should be integers

$beta^2 >= beta^0$

Monotonically Constrained Extreme Learning Machine

```
…

A = X @ W.T + b   # inputs before activation function
H = g(A)          # hidden activation
Gp = gprime(A)    # derivative of g at A

m = gp.Model("ELM_monotone_sparse")

beta = m.addMVar(shape=n_hidden, lb=-GRB.INFINITY, name="beta")
residual = H @ beta - y
obj = residual @ residual

m.setObjective(obj, GRB.MINIMIZE)

for k in inc_features:
   coeff = Gp * W[:, k]
   m.addConstr(coeff @ beta >= 0.0, name=f"monotone_inc_k{k}")

for k in dec_features:
   coeff = Gp * W[:, k]
   m.addConstr(coeff @ beta <= 0.0, name=f"monotone_dec_k{k}")

m.optimize()
```





How infomed ML will im

Overcoming Algorithm Aversion with Transparency: Can Transparent Predictions Change User Behavior?

Research Paper

First Author[1], Second Author[1], Third Author[1,2], and Fourth Author[2]

[1] Institution, Department, City, Country
{first.author,second.author,third.author}@example.com
[2] Other Institution, Other Department, Other City, Other Country
{third.author,fourth.author@anotherexample.com}

Abstract. Previous work has shown that allowing users to adjust a machine learning (ML) model's predictions can reduce aversion to imperfect algorithmic decisions. However, these results were obtained in situations where users had no information about the model's reasoning. Thus, it remains unclear whether interpretable ML models could further reduce algorithm aversion or even render adjustability obsolete. In this paper, we replicate a well-known study that examines the effect of adjustable predictions on algorithm aversion and extend it by introducing an interpretable ML model that visually reveals its decision logic. Through a pre-registered user study with 280 participants, we investigate how transparency interacts with adjustability in reducing aversion to algorithmic decision-making. Our results replicate the adjustability effect, showing that allowing users to modify algorithmic predictions mitigates aversion. Transparency's impact appears smaller than expected and was not significant for our sample. Furthermore, the effects of transparency and adjustability appear to be more independent than expected.

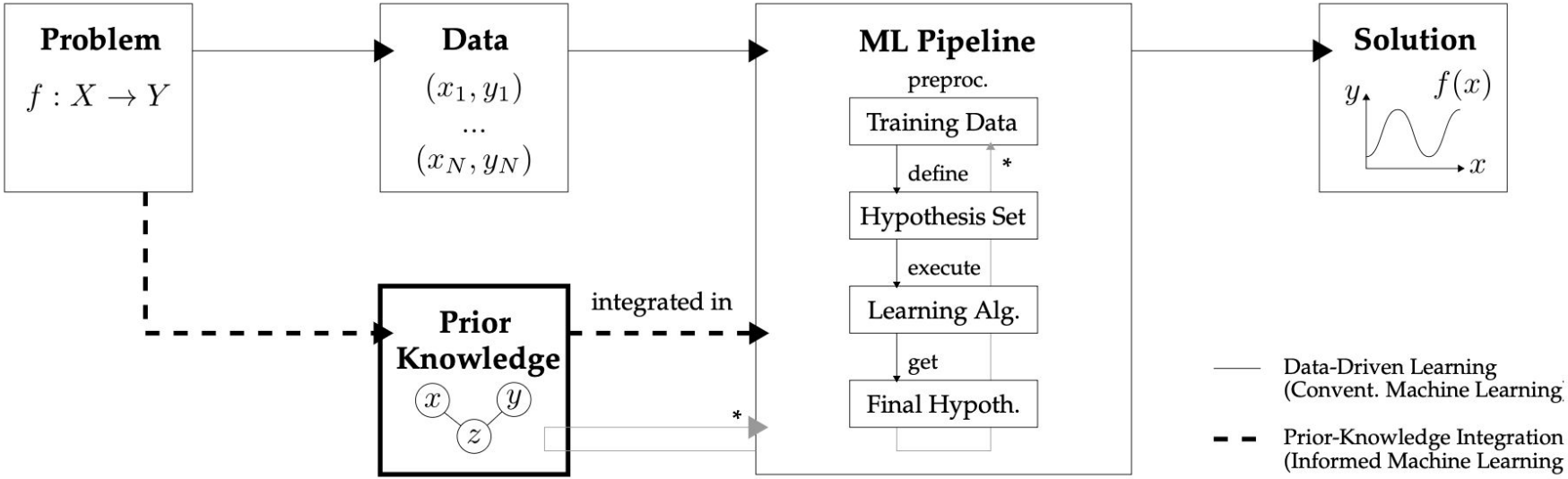
Machine Learning – nd Survey of Integrating into Learning Systems

arina Beckh, Bogdan Georgiev, Sven Giesselbach, nika Pick, Rajkumar Ramamurthy, Michal Walczak, ickhage and Jannis Schuecker

its when dealing with insufficient training data. A potential rocess which leads to the notion of *informed machine learning.* his field. We provide a definition and propose a concept for shes it from conventional machine learning. We introduce a ning approaches. It considers the source of knowledge, on this taxonomy, we survey related research and ogic rules, or simulation results can be used in ny uncovers key methods in the field of informed

nformed, Hybrid, Neuro-Symbolic, Survey, Taxonomy.

Prior Knowledge Integration:



(2021) Informed Machine Learning – A Taxonomy and Survey of Integrating Prior Knowledge into Learning Systems. von Rueden et al. 10.1109/TKDE.2021.3079836

Development and Evaluation Framework

- Employ Interpretable generalized additive neural networks (IGANN) as the ML model
- Support for constraints like monotonicity, feature importance etc will be added to the algorithm.
- Develop the backend to enable real-time computation of the modified IGANN algorithm
- Design the user interaction environment and questionnaire
- Conduct the experiment and assess user feedback

Optimal Decision Trees

Anastasia Samets

Researched Types

01

Quant-BnB

02

ConTree

03

OCT

04

OCT-H

05

BendersOCT-5

06

BendersOCT-10

Quant-BnB

Research Paper Link:
https://arxiv.org/pdf/2206.11844

01 Approach

Branch and Bound directly supporting continuous features. 2022 algorithm.

02 Scalability and Depth

Depth limit <= 3. Scalability moderate-large.

03 Usages

Useful for optimal but shallow trees on continuous data.

ConTree

Research Paper Link:
https://arxiv.org/pdf/2501.07903

01 Approach

Dynamic programming with Branch and Bound, with 3 new pruning techniques. Also utilizes a depth 2 subroutine. Handles continuous features directly. 2025 algorithm.

02 Scalability and Depth

Supports trees of depth >=4. Faster than Quant-BnB (10x). Large scalability.

03 Usages

Use when working with continuous features and need deep trees.

OCT & OCT-H

Research Paper Link:
https://dspace.mit.edu/bitstream/handle/1721.1/110328/10994_2017_5633_ReferencePDF.pdf?sequence=2&isAllowed=y

01 Approach

Data must be pre-binarized. Utilize MIP formulations to globally optimize. OCT-H supports multivariate splits.

02 Scalability and Depth

OCT-H smaller depths than OCT, OCT <=3-4. OCT and OCT-H are moderate scalability but OCT-H is more limited.

03 Usages

Use when operating on binary encoded features and require provable optimality guarantees (in MIP).

BendersOCT-5 and BendersOCT-10

Research Paper Link:
https://arxiv.org/pdf/2103.15965

01 Approach

Utilizes MIP with flow cuts and Benders algorithm. Both require binarization. 5 versus 10 is the number of discretization buckets. 5 used more often because more efficient.

02 Scalability and Depth

5's depth is <=3-4 and its scalability is moderate.

03 Usages

Use when you have binary encoded features and require providing optimality guarantees (in MIP).

OCT Code Example

For fun, that's the code on the right!

Breakdown:

- OCT approach where you use mixed integer programming and solve via Gurobi optimizer
- Goal: find the best binary decision tree with a depth limit that: minimizes classification error + penalty for tree complexity
- OCT: can only split at one feature per branching, via threshold
    - <= threshold goes left, > threshold goes right
- MIP: integers and continuous values included
    - Here: threshold tuning utilizes continuous values
- Linear objective

```
Globally optimal binary decision tree, solved with Gurobi
==========================================================
 * depth:       maximum tree depth (≤ 3 recommended for demo)
 * alpha:       complexity-penalty for #leaves (CART-style cost-complexity pruning)
 * X, y :       numpy arrays  (y must be {0,1})
 * Returns:     fitted leaf predictions and a predict() helper
"""
import numpy as np
import gurobipy as gp
from gurobipy import GRB


def optimal_decision_tree(X, y, depth=2, alpha=0.01, time_limit=None, verbose=False):
    n, p = X.shape
    assert set(y) <= {0, 1}, "Only binary labels supported"

    # ---- bookkeeping on the tree geometry ----------------------------------
    int_nodes = range(2**depth - 1)          # internal nodes: 0 .. 2^d − 2
    leaves    = range(2**depth, 2**(depth + 1))  # leaves:      2^d .. 2^{d+1} − 1

    # Helper: for (node i, leaf ℓ) return True/False whether ℓ is in the *left* child of i
    def is_left(i, ell):
        while ell > i:           # climb from leaf up to root
            parent = (ell - 1) // 2
            if parent == i:
                return ell == 2 * i + 1   # left child index
            ell = parent
        return False  # should never happen

    # ---- model -------------------------------------------------------------
    m = gp.Model("OCT")
    if time_limit:
        m.setParam("TimeLimit", time_limit)
    m.setParam("OutputFlag", int(verbose))

    # 1. Split-selection variables
    a = m.addMVar((len(int_nodes), p), vtype=GRB.BINARY, name="a")   # a[i,j] = 1  ⇒ node i
splits on feature j
    m.addConstr(a.sum(axis=1) == 1, name="one_feature_per_node")

    # 2. Split thresholds
    t = m.addMVar(len(int_nodes), lb=-GRB.INFINITY, name="t")

    # 3. Leaf prediction (c_ℓ ∈ {0,1})
    c = m.addMVar(len(leaves), vtype=GRB.BINARY, name="c")

    # 4. Assignment of each sample to exactly one leaf
    z = m.addMVar((n, len(leaves)), vtype=GRB.BINARY, name="z")
    m.addConstr(z.sum(axis=1) == 1, name="one_leaf_per_obs")

    # 5. Path-compatibility constraints via big-M
    #
    # For every (obs k, int-node i, leaf ℓ):
    #   If ℓ is on the *left* of i and z_{kℓ}=1, then (x_{k,feat(i)} ≤ t_i)
    #   If ℓ is on the *right*           ...    z_{kℓ}=1, then (x_{k,feat(i)} > t_i)
    # We linearise with big-M and the feature-selector a[i, j].
    #
    M = (X.max(axis=0) - X.min(axis=0)).max() + 1.0   # safe global Big-M
```

```
for i in int_nodes:
    for ell_idx, ell in enumerate(leaves):
        left_side = is_left(i, ell)
        # Build \sum_j a_{i,j} x_{k,j}  − this is linear in decision variables
        for k in range(n):
            # linear expr: Σ_j (a_{i,j} * x_{k,j})
            lhs = (X[k, :] @ a[i, :])
            if left_side:
                # x ≤ t   ⇒   lhs - t_i ≤ M(1 - z_{kℓ})
                m.addConstr(lhs - t[i] <=  M * (1 - z[k, ell_idx]),
                            name=f"path_left_{i}_ell{ell}_k{k}")
            else:
                # x > t   ⇒   lhs - t_i ≥ ε - M(1 - z),   using ε>0 to rule out equality
                eps = 1e-4
                m.addConstr(lhs - t[i] >= eps - M * (1 - z[k, ell_idx]),
                            name=f"path_right_{i}_ell{ell}_k{k}")


    # 6. Misclassification indicator for each obs/leaf combo
    mis = m.addMVar((n, len(leaves)), vtype=GRB.BINARY, name="mis")
    for ell_idx, _ in enumerate(leaves):
        # If obs k assigned to ℓ and y_k ≠ c_ℓ  ⇒  mis = 1
        # Linearise:   mis_{kℓ} ≥ z_{kℓ} - (1 - |y_k - c_ℓ|)
        # Equivalent compact pair of constraints:
        for k in range(n):
            # y_k (0/1) so |y_k - c_ℓ| equals y_k*(1-c_ℓ) + (1-y_k)*c_ℓ
            m.addConstr(mis[k, ell_idx] >= z[k, ell_idx] * (y[k] - c[ell_idx]),
                        name=f"mis1_k{k}_ell{ell_idx}")
            m.addConstr(mis[k, ell_idx] >= z[k, ell_idx] * (c[ell_idx] - y[k]),
                        name=f"mis2_k{k}_ell{ell_idx}")

    # 7. Objective:   total misclassifications  +  α · (#leaves used)
    obj = mis.sum() + alpha * c.sum()
    m.setObjective(obj, GRB.MINIMIZE)

    m.optimize()

    # ------------------------------------------------------------------------
    # helper to predict on new data
    def predict(Xnew):
        Xnew = np.asarray(Xnew)
        nnew = Xnew.shape[0]
        # extract learned split structure
        selected_feat = np.argmax(a.X, axis=1)
        thresholds    = t.X
        leaf_pred     = c.X.round().astype(int)   # 0/1

        preds = np.zeros(nnew, dtype=int)
        for kk in range(nnew):
            node = 0
            for d in range(depth):
                j = selected_feat[node]
                if Xnew[kk, j] <= thresholds[node]:
                    node = 2 * node + 1      # go to left child
                else:
                    node = 2 * node + 2      # right child
            leaf_idx = node - (2**depth)
            preds[kk] = leaf_pred[leaf_idx]
        return preds

    return predict, m  # returning the Gurobi model as well is handy for introspection


# ----------------------------------------------------------------------------#
# Small demonstration on a toy data-set
if __name__ == "__main__":
    np.random.seed(0)
    m_samp, n_feat = 120, 4
    X_demo = np.random.randn(m_samp, n_feat)
    y_demo = (X_demo[:, 0] + X_demo[:, 1] > 0).astype(int)  # linearly separable

    pred_fn, model = optimal_decision_tree(X_demo, y_demo,
                                           depth=2, alpha=0.001,
                                           time_limit=30, verbose=True)

    print("\n--- Training error:", (pred_fn(X_demo) != y_demo).mean())
    # examine optimal splits
    print("Selected features by node:", np.argmax(model.getVarByName("a").X.reshape(-1, n_feat), axi
    print("Thresholds:", model.getVarByName("t").X)
```

Possible Research Contributions

- Which constraints can we come up with? (importance, monotonic, limits, …)
- Which models can be boosted to include human informed constraints?
- Can we implement a general platform to allow users to work with IIIML models?
- How do these models scale?

Dietvorst et al. 2018: Giving people the ability to slightly adjust an algorithm's output significantly increases their willingness to use it. We can build upon that, e.g., in these directions:

H1: Informing ML models increases the users' willingness to use it

H2: Informing ML models by experts increases model performance after (slight) distribution shits

H3: Users better recall model behavior over time when they informed the ML model

* I did not talk about post-processing, but similar questions can be asked there.